Section D — Joins & Relationships Documentation

This section focuses on understanding relationships between multiple tables and combining data using different types of SQL JOINs.

The main objective of this section is to practice:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- Understanding Foreign Keys
- Maintaining data relationships
- Handling relational integrity

Joins are important because real-world databases store related data in separate tables.

Database Selection

Query

```
USE shopease_db;
```

Explanation

This command selects the ShopEase database before running JOIN queries.

It ensures all operations are executed on the correct database.

Q19. Display order details with customer names using INNER JOIN

Query

```
SELECT
    o.order_id,
    o.order_date,
    c.first_name,
    c.last_name,
    o.total_amount
```

FROM orders o

INNER JOIN customers c

ON o.customer_id = c.customer_id;

Explanation

This query combines data from two tables:

- orders
- customers

using INNER JOIN.

How it works:

- orders table is given alias o
- customers table is given alias c
- Both tables are joined where customer_id matches

Only matching records from both tables are returned.

This means:
If an order has a valid customer, it will appear in the result.

Purpose

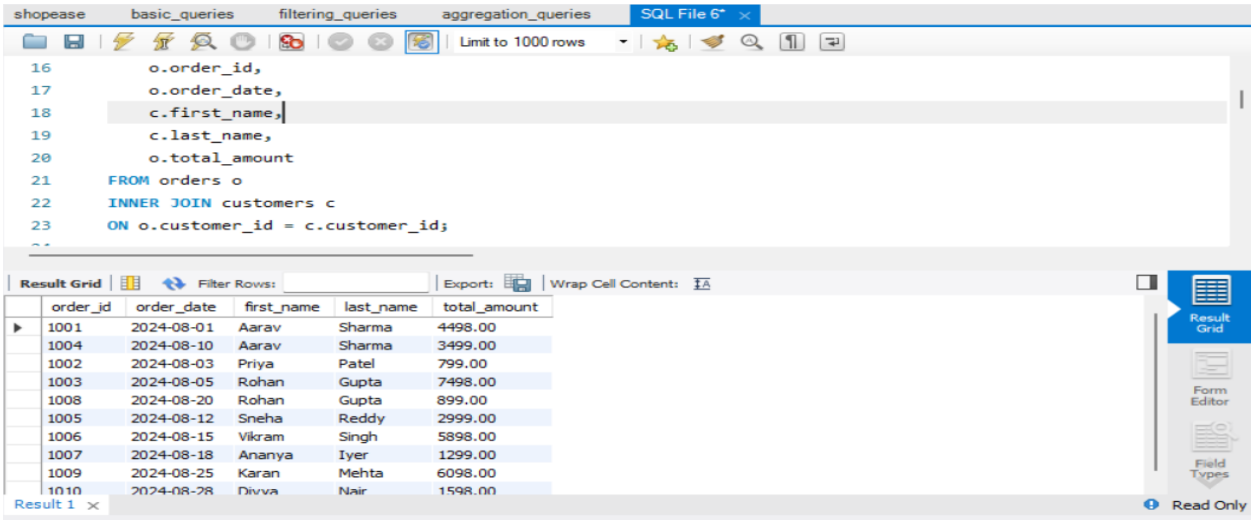
Helps identify which customer placed which order.

Expected Output

Displays:

- Order ID
- Order Date
- Customer First Name
- Customer Last Name
- Total Amount

Screenshot



Q20. Show all customers and their orders using LEFT JOIN

Query

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    o.order_id,
    o.order_date,
    o.total_amount
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

Explanation

This query uses LEFT JOIN.

How it works:

- Returns all rows from the left table (customers)

- Returns matching rows from the right table (orders)
- If no matching order exists, order columns show NULL

This ensures no customer is missed.

Purpose

Useful for checking:

- customers who have placed orders
- customers who have not placed any orders

Expected Output

Shows all customers.

If a customer has no order:

- order_id = NULL
- order_date = NULL
- total_amount = NULL

Q21. Join three tables: orders → order_items → products

Query

```
SELECT
   o.order_id,
   p.product_name,
   oi.quantity,
   oi.unit_price,
   oi.discount_pct
FROM orders o
INNER JOIN order_items oi
ON o.order_id = oi.order_id
INNER JOIN products p
ON oi.product_id = p.product_id;
```

Explanation

This query combines three related tables.

Relationship flow:

orders → order_items → products

Step-by-step:

1. orders joins with order_items using order_id
2. order_items joins with products using product_id

This gives product-level details for each order.

Purpose

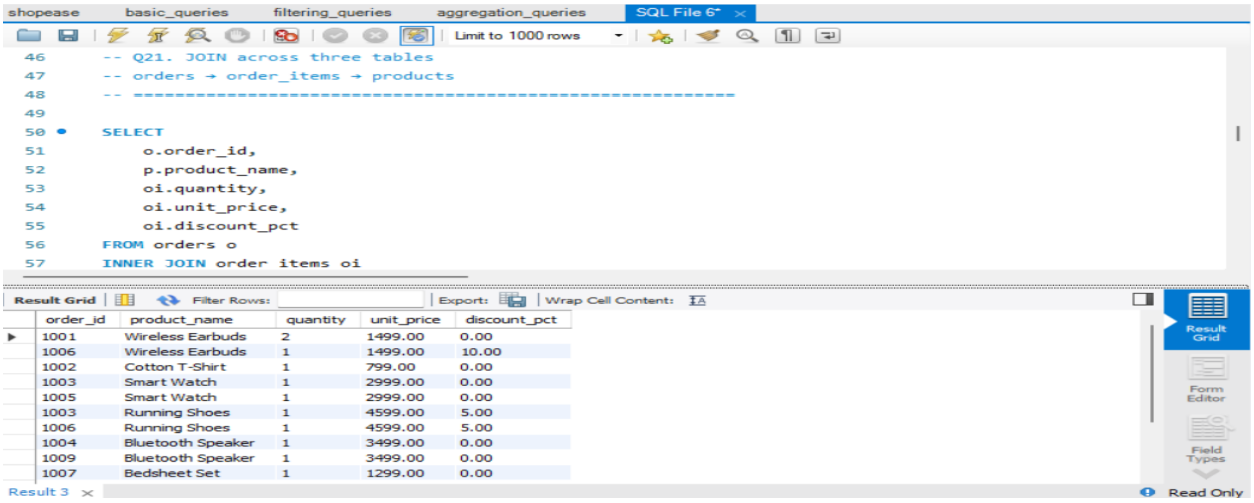
Helps analyze what products were purchased in each order.

Expected Output

Displays:

- Order ID
- Product Name
- Quantity
- Unit Price
- Discount Percentage

Screenshot



Q22. Difference between LEFT JOIN and RIGHT JOIN

LEFT JOIN Query

```
SELECT
    c.customer_id,
    c.first_name,
    o.order_id
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

Explanation

Returns:

- All customers
- Matching orders

If no order exists:
order_id becomes NULL.

LEFT JOIN prioritizes the left table.

RIGHT JOIN Query

```
SELECT
    c.customer_id,
    c.first_name,
    o.order_id
FROM customers c
RIGHT JOIN orders o
ON c.customer_id = o.customer_id;
```

Explanation

Returns:

- All orders
- Matching customers

If no customer exists (rare because of foreign key), customer data becomes NULL.

RIGHT JOIN prioritizes the right table.

Difference Summary

LEFT JOIN:
Keeps all rows from left table.

RIGHT JOIN:
Keeps all rows from right table.

FULL OUTER JOIN

Explanation:

FULL OUTER JOIN returns:

- All rows from left table
- All rows from right table
- Matched and unmatched records

Use case:

When you want all customers and all orders regardless of matching.

Note:

MySQL does not directly support FULL OUTER JOIN.

Q23. Foreign Key Relationships

Foreign Keys in this schema

1. orders.customer_id → customers.customer_id
2. order_items.order_id → orders.order_id
3. order_items.product_id → products.product_id

These keys connect tables and maintain relationships.

Checking foreign keys

Query

DESCRIBE orders;

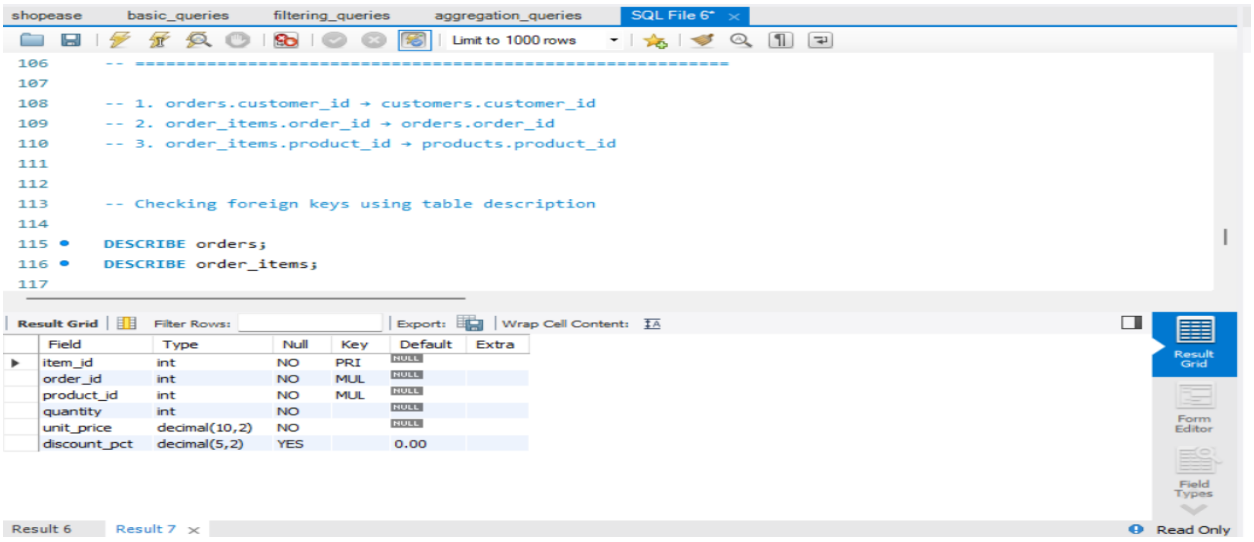
DESCRIBE order_items;

Explanation

The DESCRIBE statement shows table structure and key information.

It helps verify foreign key columns.

Screenshot



Invalid Foreign Key Example

Query

INSERT INTO orders VALUES

(1011, 999, '2024-08-30', 'Pending', 1500.00);

Explanation

This query tries to insert an order for customer_id = 999.

But customer 999 does not exist in the customers table.

Because of foreign key constraints, MySQL rejects this insertion.

This prevents invalid relationships.

Expected Error

Cannot add or update a child row:
a foreign key constraint fails

Key Concepts Covered

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- Table Aliases
- Foreign Keys
- Relationship Mapping
- Data Integrity

Summary

In this section, multiple tables were connected using JOIN operations.

The queries demonstrated how relational databases store data separately and connect it using keys.

Understanding joins and foreign keys is essential for retrieving meaningful combined data from multiple related tables.